

RAI-853: Advanced Robotics with ROS

Lec 02b: What is ROS

Instructor:

Dr. Khawaja Fahad Iqbal

Deputy HoD (Department of Robotics & AI, SMME, NUST)

Scientific Director (Intelligent Robotics Lab, NCAI)



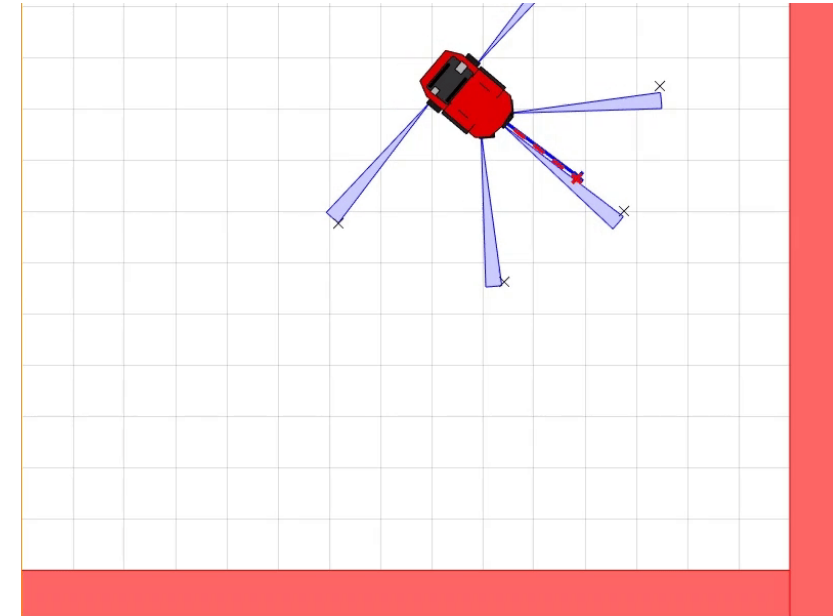
- **Why ROS?**
- **What is ROS?**
- **When to Use ROS?**
- **ROS 1 versus ROS 2**
- **Prerequisites For Starting With ROS 2**

- **Why ROS?**
- What is ROS?
- When to Use ROS?
- ROS 1 versus ROS 2
- Prerequisites For Starting With ROS 2

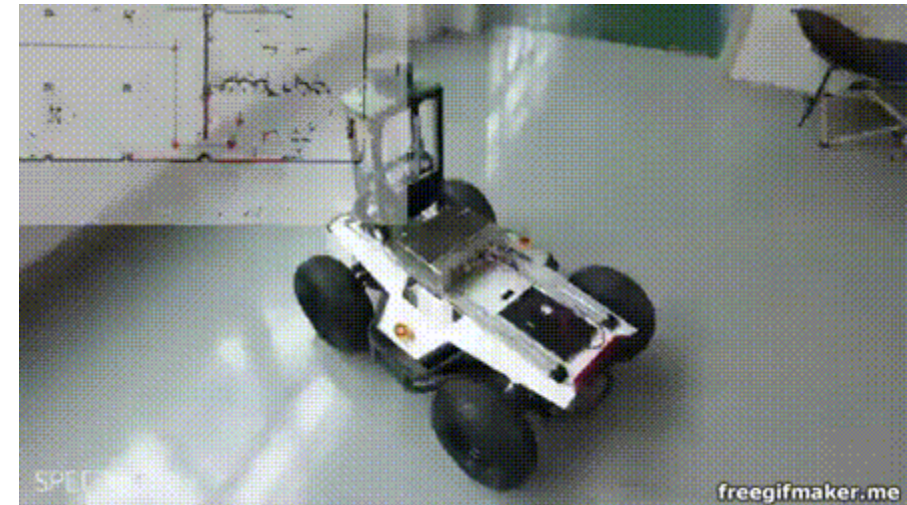
- Let's start with a big problem that occurs often in robotics.
- Suppose that you just got a new project at your job, and **you have to develop a robotics application**, or you are doing a new research thesis.
- Any project or thesis will have a specific duration, from a few months to a few years.
- Now, what will happen?



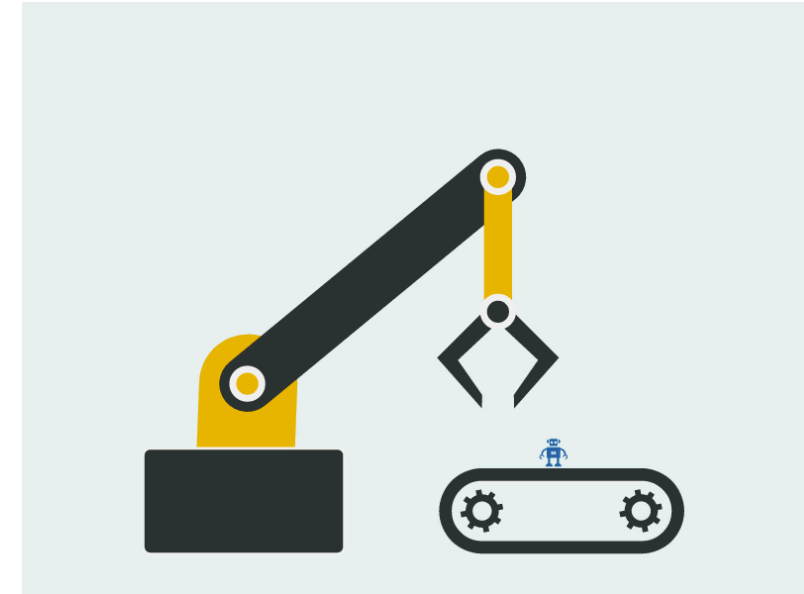
- You start to design the robotics system you need for your project and soon realize all the existing solutions you found don't match what you need.
- After a few weeks, you finalize the specifications, and you start building your robot.
- A few months later, **you're still developing the basic software** for robot control and navigation.
- You underestimated how much time it would take you to just get the robot running.



- After one year, you realize that all you've done so far is build a robotics system, and **you still haven't started the core functionality** of your application or research. Now is the time to hurry.
- You finish the robot as well as you can, make some shortcuts, and publish your paper or present that prototype.
- In the best-case scenario, you could also share your code with an open-source license so that other people can use it, but probably not directly as it's just code for your own need, **not a complete framework** or library with modular components, documentation, and so on.



- Then, you move on to a new project, new job, and new research. Somebody else will take your place, read your code, and realize that it doesn't help them build their application. Hence, they need to start from scratch.
- What just happened here is that you reinvented the wheel. The next person will repeat the same circle. This is the number one reason why ROS was created: **to stop you from reinventing the wheel** anytime you need to create a robot.
- Just like you have open-source frameworks, tools, and environments to develop websites or mobile applications, why not do the same for robotics?



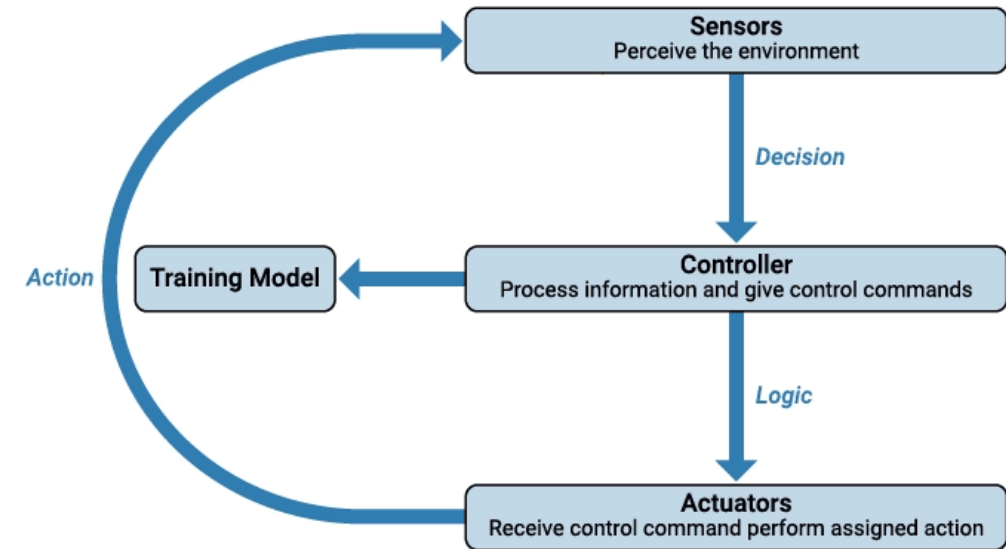
- This is the philosophy behind ROS: **to provide a standard for robotics applications** that you can use on any robot.
- After you learn ROS, you can spend less time on the basics and focus on the key functionalities you want to add instead.
- You can program new robots in no time, join existing projects, and easily collaborate with a team.



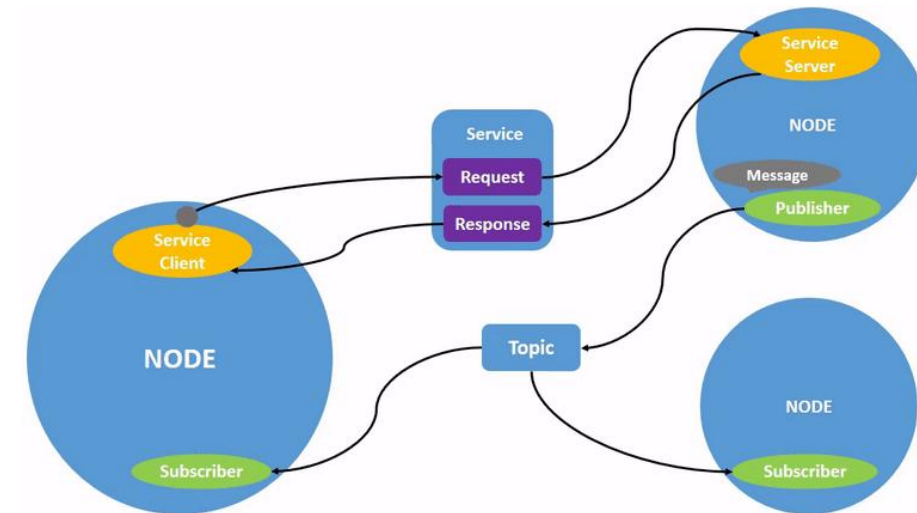
ROS

- Why ROS?
- **What is ROS?**
- When to Use ROS?
- ROS 1 versus ROS 2
- Prerequisites For Starting With ROS 2

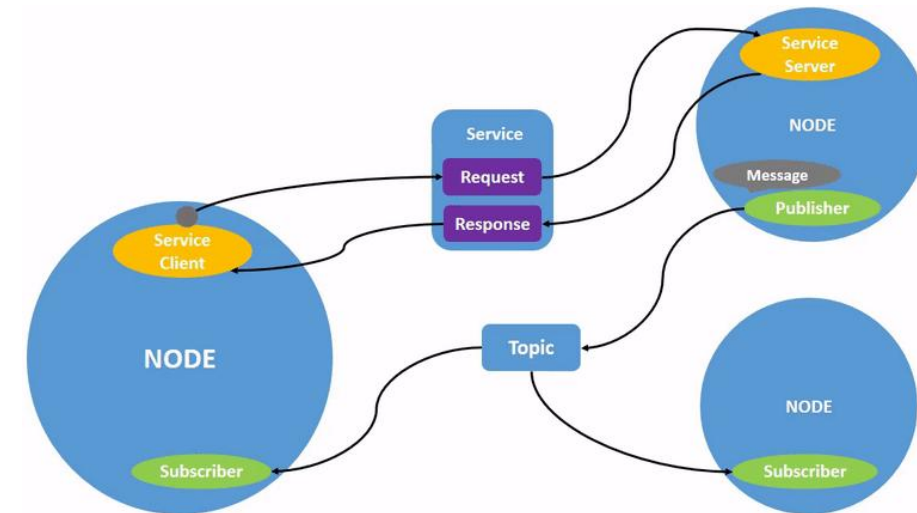
- ROS is hard to define because it's not just one thing. It is hard for anyone to truly understand what it is until they start to understand how to write code with it.
- One thing we can start with is what ROS is not. **ROS is not an operating system.**
- It is a combination of four main parts:
 - Framework
 - Set of tools
 - Plug-and-play plugins
 - An online community



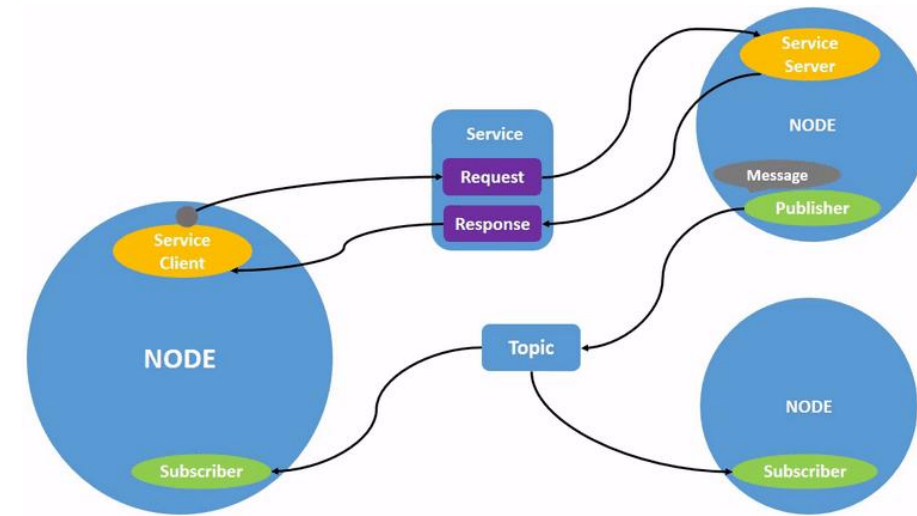
- In ROS, you need to create packages, and then write programs inside those packages (**nodes**).
- There is a specific way to create and write them, as well as create tools to build and use them.
- After you've created a few projects, any new project is going to be easier and faster to set up.



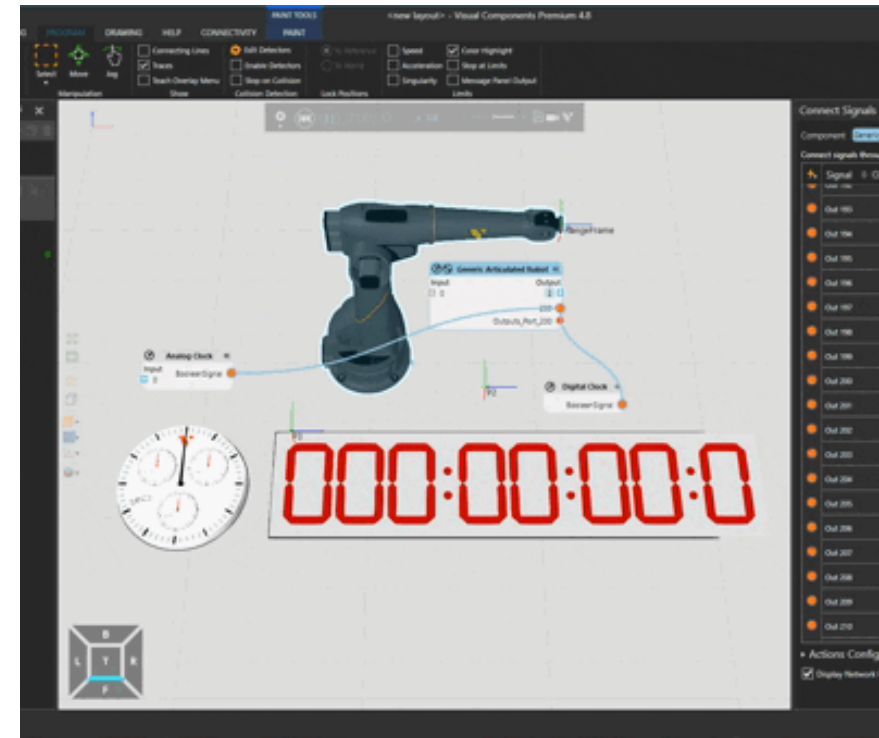
- Since everyone is following the same set of rules, you can more easily work in a team or understand and use the code written by others.
- As a direct consequence of using this framework, you get access to plumbing, which means that the **underlying communication** between the nodes is managed for you.
- This will save you a lot of development time, and you also don't need to learn how to do it yourself (and thus, reinvent the wheel).



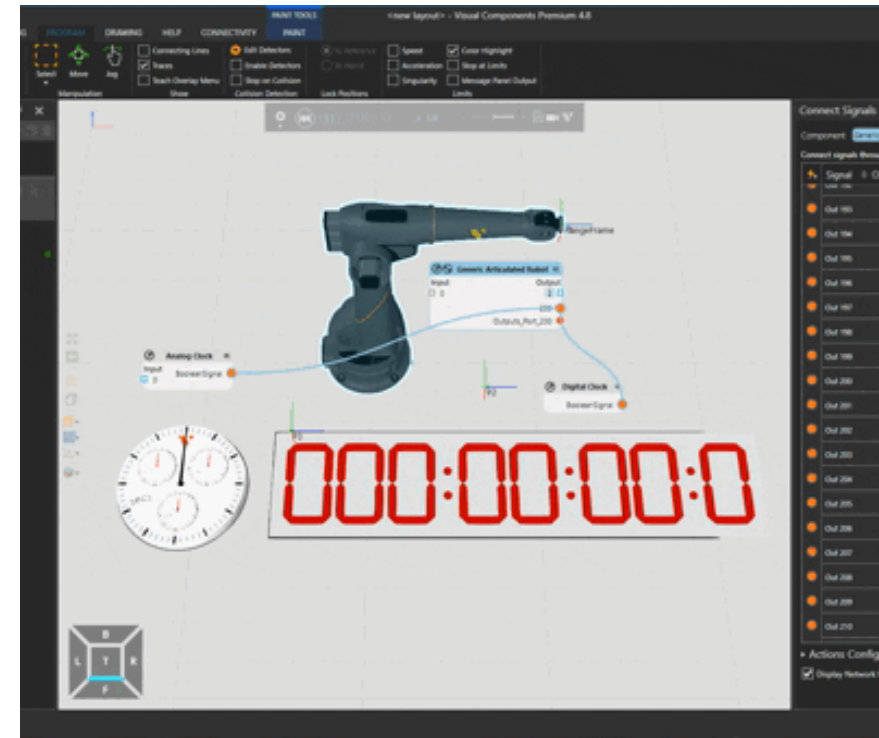
- To sum it up, with ROS, you can easily separate your application into different sub-programs (called nodes). The communication between nodes is handled for you.
- You can easily test one component, and if this component fails, it will not affect the other running components.
- **ROS is a modular framework.**



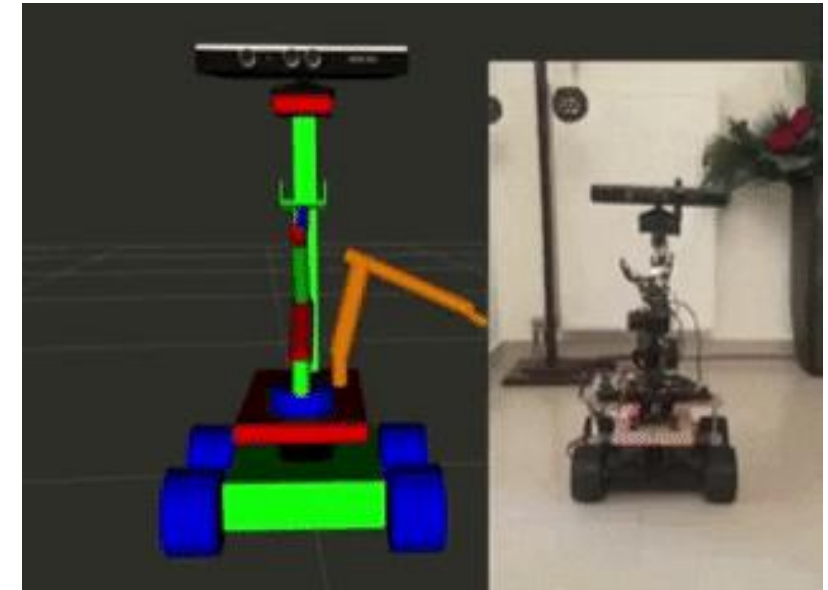
- ROS comes with a set of tools that allow you to develop faster.
- Among them, you can find command line tools to **build** the application, **introspection** tools to monitor the flow of communication, **logging** functionalities, plots, and more.
- You also get **3D visualization** tools to see what your robot is doing, and even a complete simulator using real physics, named **Gazebo**, so that you can work on a realistic simulation before trying out your robot.



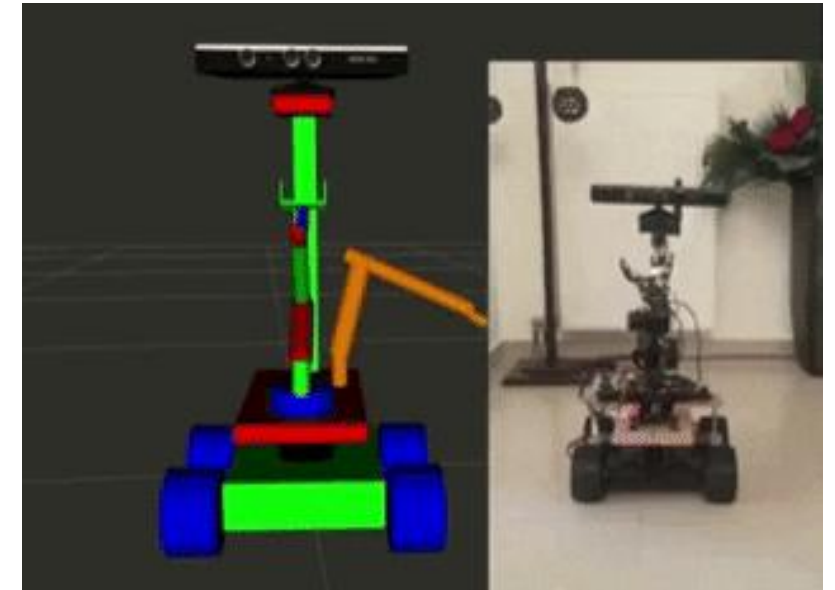
- As an example of how useful it can be, there is one (called **bags**) that allows you to save communication streams so that they can be replayed later.
- Let's say you build a mobile robot, and you need to test the robot outside when it's raining, then continue to develop the software while taking the rain into account.
- You probably won't have rain every day, or you won't even have access to the robot any time you want. With this tool, **you can run the experiment once, save the data, and replay it later** to develop your application for a specific set of conditions.



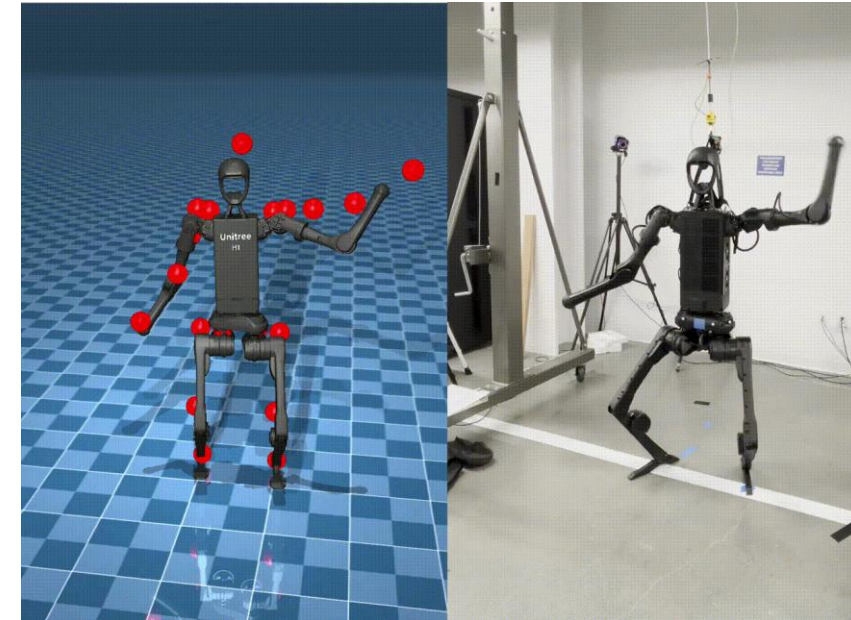
- Imagine two common scenarios:
 - You develop a mobile robot and need the robot to navigate autonomously in a dynamic environment.
 - You develop a six-axis robotic arm and want to create motion planning to perform a smooth movement on all axes.
- This looks quite complex and involves understanding and implementing several algorithms, as well as writing well-optimized and efficient code.
- This is where you would probably have to reinvent the wheel and waste lots of precious time.



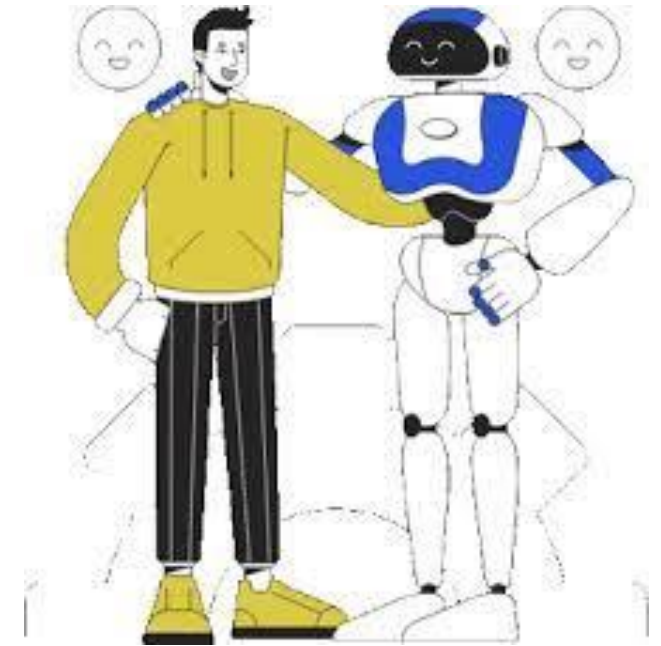
- For those two scenarios, you can find existing plugins that do the job for you.
- All you need to do is install the plugins and configure your robot to make it compatible.
- Of course, this is easier said than done, but the workload can be counted in days/weeks, not months/years.
- And once again, once you know how to use those plugins, your next project will take much less time.



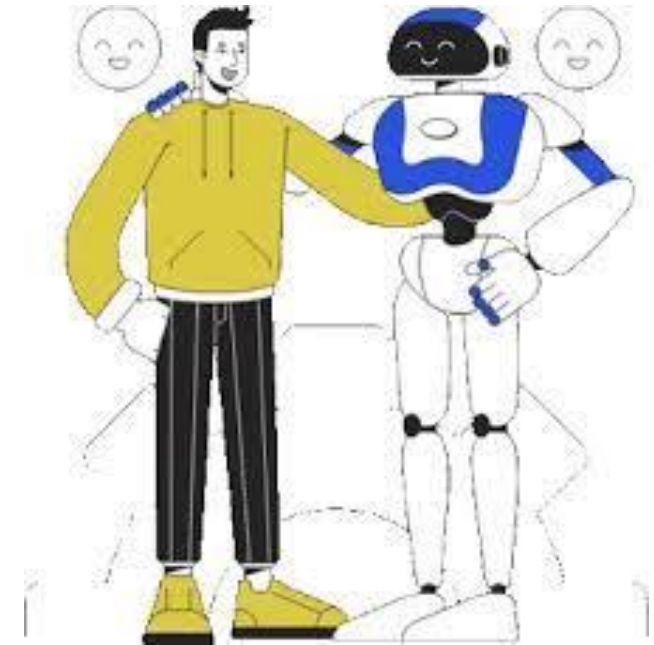
- There are many plugins that you can use. Some are quite simple, while others involve a collection of plugins and are also called frameworks or stacks.
- Your job as a ROS developer is to glue all those components together, and maybe create new components for functionalities that are not developed yet.



- ROS is an open-source project with a permissive license.
- You can use ROS in a commercial product without having to redistribute your code.
- You can find all the ROS code online, as well as the code for the plug-and-play plugins.
- Everything is easily accessible on GitHub.

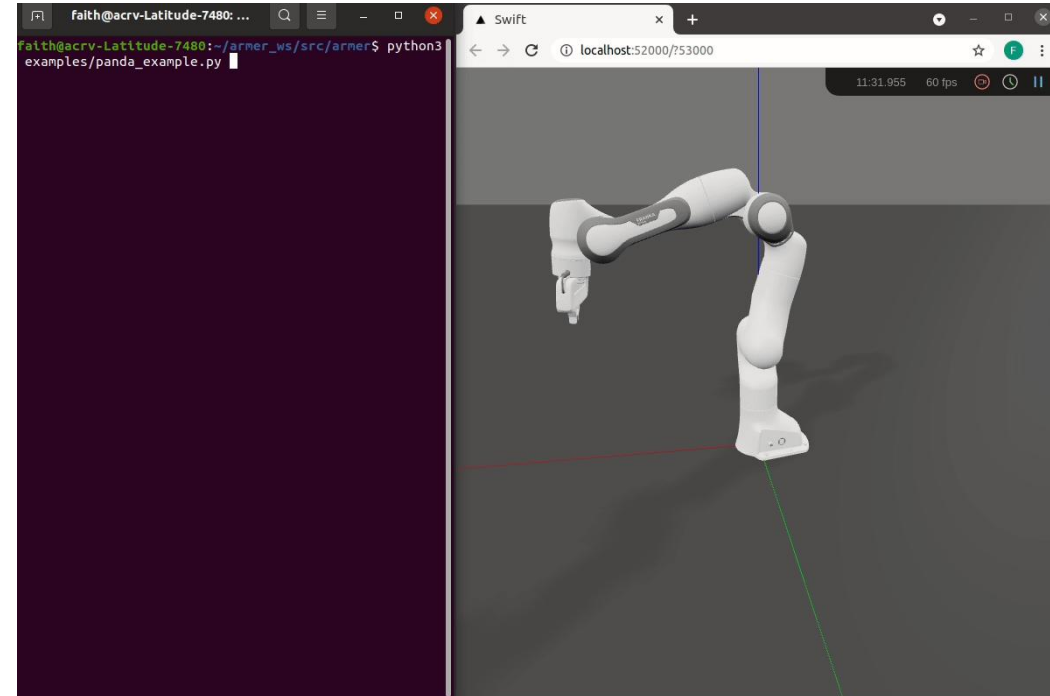


- The ROS project is also backed by an online community that you can most commonly find in the following areas:
- **Robotics Stack Exchange**
(<https://robotics.stackexchange.com/>): You can use this to ask technical questions. This is like the Stack Overflow for robotics.
- **ROS Discourse forums**
(<https://discourse.ros.org/>): Here, you can get informed about the latest developments, jobs, community projects, new ideas, and more.

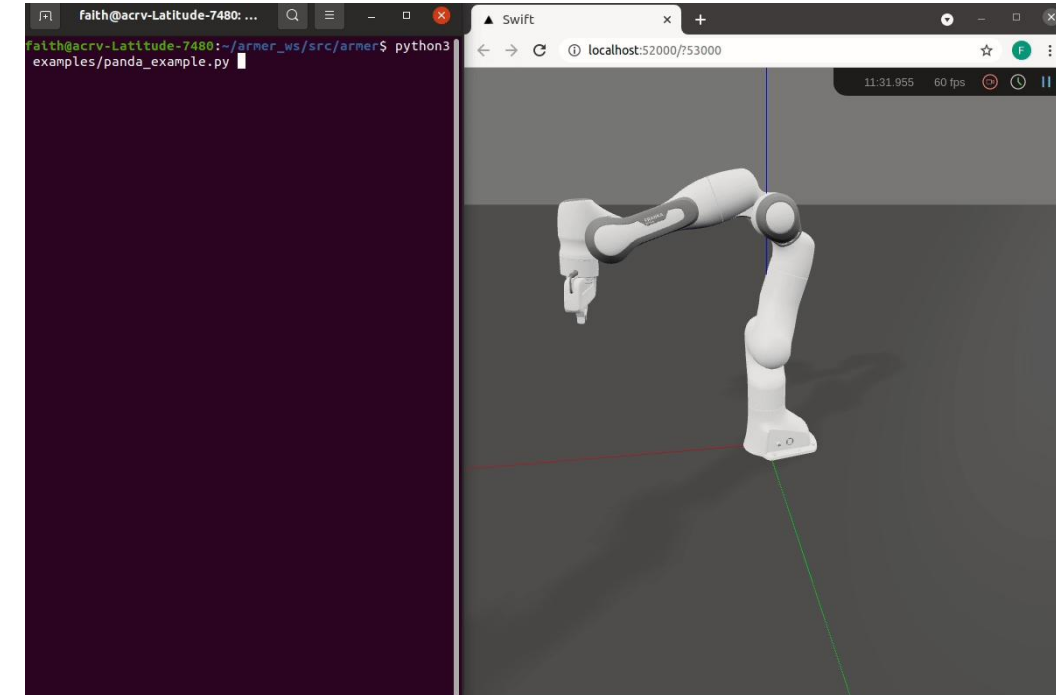


- Why ROS?
- What is ROS?
- **When to Use ROS?**
- ROS 1 versus ROS 2
- Prerequisites For Starting With ROS 2

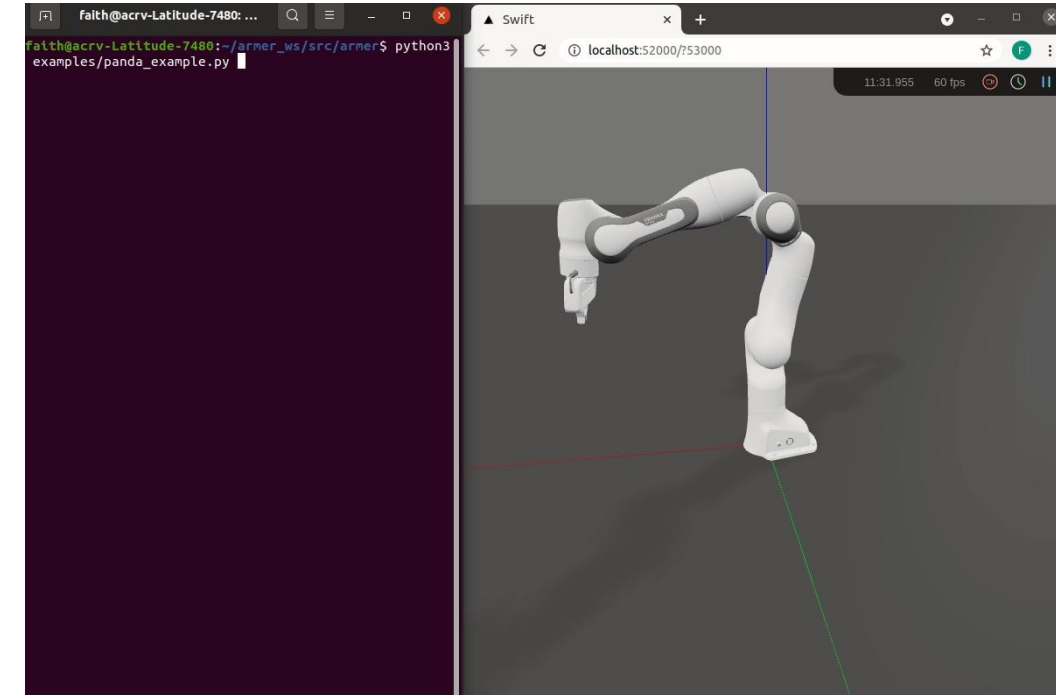
- Now that you understand a bit more what ROS is, should you use ROS whenever your project has something 'robotics' in it?
- If you need to learn ROS for your work/university, then the question is easily answered: yes, you will use ROS for your project.
- But if you must make the decision yourself, what should you do?



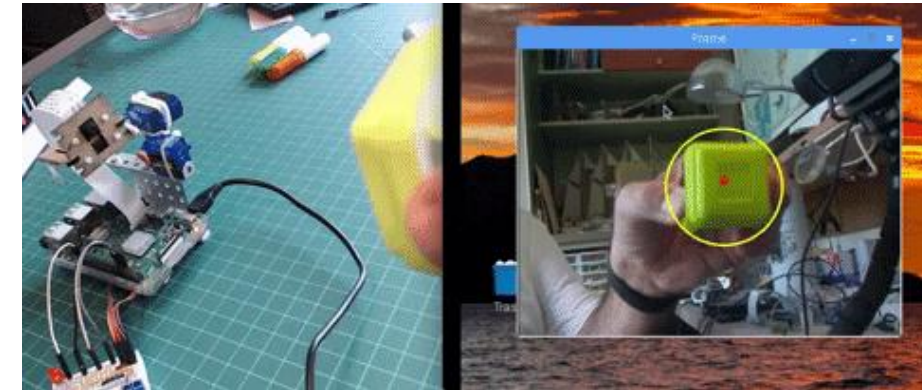
- Let's simplify robotics and say that a robotics system contains three categories of things: actuators, sensors, and controllers.
- An **actuator** is something that creates movement (for example, a motor to rotate a wheel).
- A **sensor** will read data from the environment (for example, a camera, laser scan, or temperature sensor).



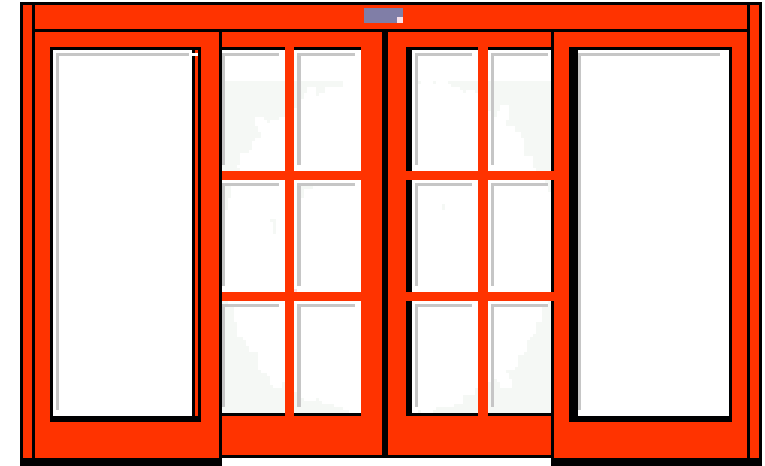
- A **controller** is something that is in between: it takes the data available from one or multiple sensors (input) and, through an algorithm, creates a command for the actuators of the robot (output). In a way, the controller is the brain or one of the brains of the robot.
- For very simple applications, when you just have a few sensors and actuators, you might not need ROS.



- Here are a few examples where ROS isn't needed:
- **First Example:** You just need to take a picture from a camera when a user presses a button, using a Raspberry Pi board, and send this picture to a web server.
- There's no need to use ROS—you can just combine a few Python libraries in a script, and you're done.
- Using ROS here would be a good example of over-engineering (unless you do this for learning purposes).



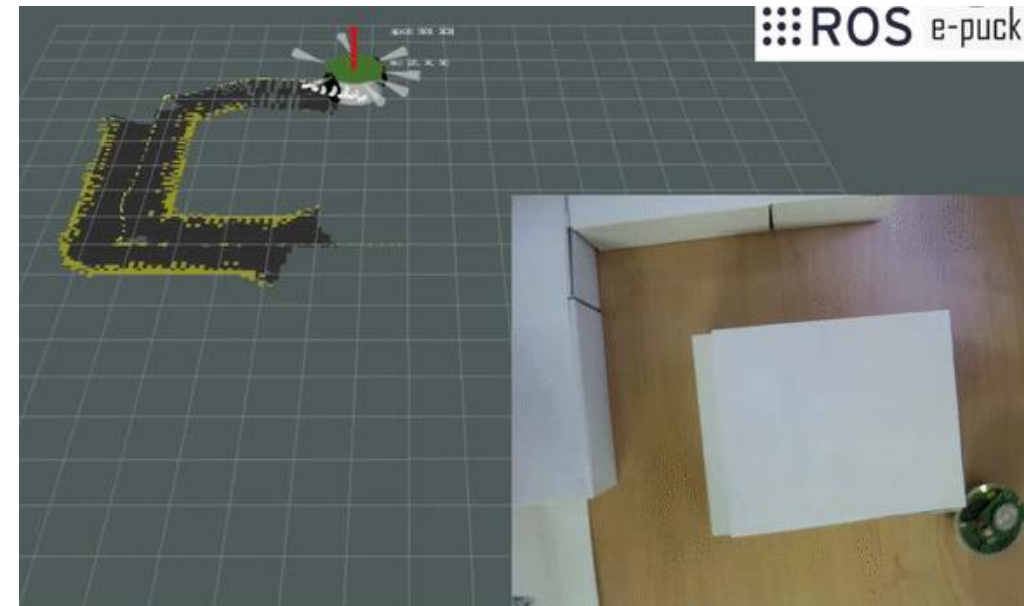
- **Second Example:** You have to use a servo motor to open/close a door when a movement is detected, using an infrared sensor.
- This is a very simple application that can easily be programmed using a basic microcontroller board—and you can do a quick prototype with a board such as Arduino.



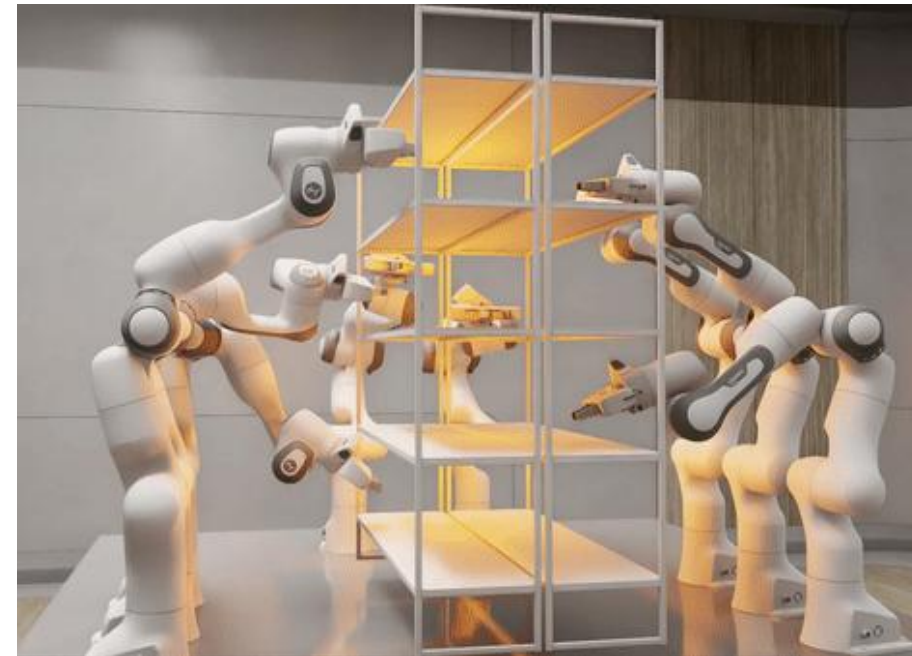
- **Third Example:** You have built a simple robot with two wheels and an infrared sensor, and you want to make the robot follow a line.
- This is a typical project that's given to students in engineering school, and a simple algorithm on an Arduino board will do.



- Now, let's consider some examples where ROS is needed:
- **First Example:** You have a new mobile robot with two wheels and a laser scan, and you want to read data from the laser scan, map the environment, make the robot move autonomously, and control the two wheels accordingly.
- On top of that, you want to simulate the robot in 3D with real physical properties.



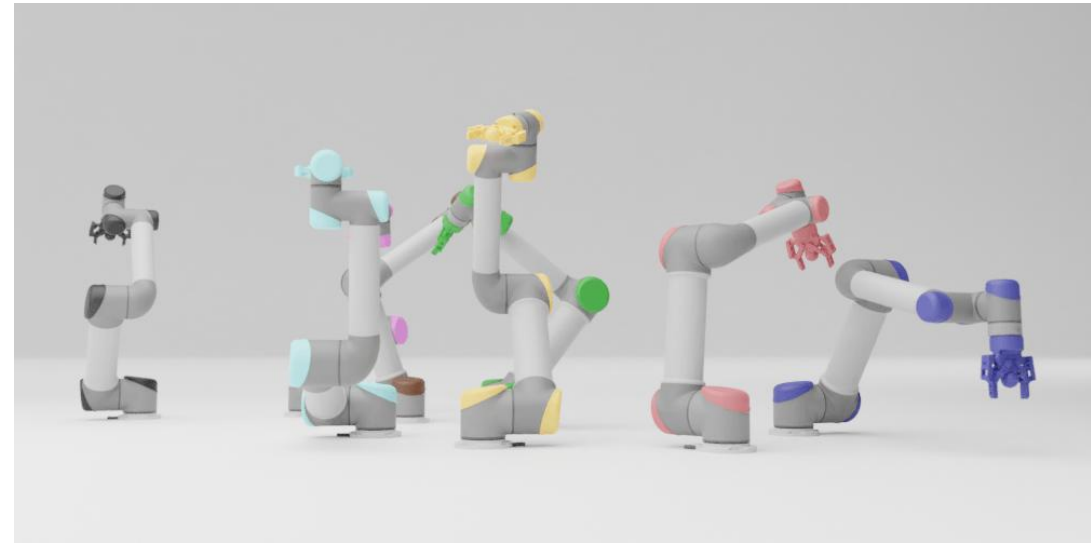
- This is when ROS is going to become very handy.
- Not only will it help you to make all the components work together, but you can also use existing algorithms for path planning (through a ROS plugin) and simulation.
- **Second Example:** You need to create a system that contains a six-axis robotic arm, or even multiple robotic arms working together, along with conveyor belts and mobile robots.



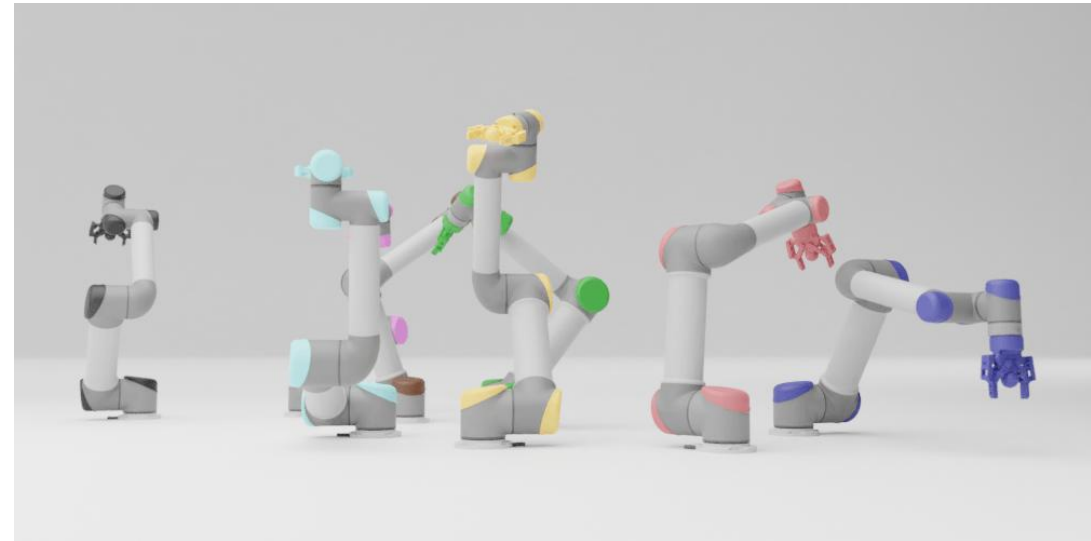
- **Third Example:** Your robotics application (not necessarily just one robot) contains lots of sensors and actuators that you want to develop separately and add them in a modular way.
- **Fourth Example:** You want to create a hardware driver for a component and make this component easy to use by other robotics developers.
- By making the component ROS-compatible, anybody who knows ROS can integrate it into their application with low effort.



- As you can see from the former examples, ROS is not always needed whenever you need to program hardware or create a robotics system.
- With the latter examples, you can see that if your system becomes more complex, if you want to easily collaborate with other developers, or if you realize that one big part of your system can be solved with one of the plug-and-play plugins, ROS may be the solution.



- Of course, it takes time to learn it and your first project will take longer to complete, but then, with more experience, you will go much faster.
- As an example, it could take less than a week for a senior ROS developer to write custom code for a robotic arm (including robot model, motion planning, and hardware control) and the same for a mobile robot with navigation capabilities (provided that the hardware already has a ROS driver).



- Why ROS?
- What is ROS?
- When to Use ROS?
- **ROS 1 versus ROS 2**
- Prerequisites For Starting With ROS 2

- ROS 1 (originally called ROS) was first developed in 2007. It quickly gained popularity and grew exponentially in the following years.
- In 2014, the ROS 2 project was announced. Simply put, ROS 1 was a bit too limited for industrial applications (lack of real-time support, safety, and so on) and was only used in research/education.
- To solve this problem, the developers decided to make ROS more “industrial friendly,” as well as make it better, thanks to all the lessons learned from the beginning of ROS.



- Now, why create ROS 2 and not just continue ROS with some new changes?
- Well, the changes were too big, and they would have completely broken compatibility with older versions.
- Thus, it was decided to create a completely new ROS from scratch and name it ROS 2.
- In 2014, ROS 2 was officially announced, and the development of the project started



- In December 2017, the first ROS 2 distribution was released, which meant that ROS 1 and ROS 2 started to co-exist.
- At this point, ROS 2 was lacking many core functionalities and plugins, making it unsuitable for serious projects. Most ROS developers were still using ROS 1.
- Years passed by and ROS 2 got more development, plugins, and more. Its popularity started to grow.



- One could say it was worth it to use ROS 2 (compared to ROS 1) starting from 2022.
- With the release of ROS 2 Humble, we had access to a long-term release that was stable, with all the major plugins and stacks working correctly, which is what you need to program a robot.
- In the meantime, it was announced that ROS 1 would end in **May 2025**. After this date, ROS 1 would still exist, but it wouldn't be supported anymore.



- It is now safe to say that ROS 2 is the way to go when developing new ROS applications.
- So, if you had previously heard about ROS 1 and ROS 2, now you know that ROS 2 is what you need to learn, and we can say that ROS 1 is a dead project.
- But is that true?



- In theory, yes, but in practice, it's (always) a bit different.
- As you probably know, several companies are using obsolete and legacy technologies.
- The reason is that updating software to a new version is often quite expensive and can also be risky.
- That's why you still see job offers from banking systems requiring skills in Cobol, a programming language from the 1960s that no one uses anymore.

Distro	Release date	Poster	Turtle, turtle in tutorial	EOL date
ROS Noetic Ninjemys (Recommended)	May 23rd, 2020			May, 2025 (Focal EOL)
ROS Melodic Morenia	May 23rd, 2018			May, 2023 (Bionic EOL)
ROS Lunar Loggerhead	May 23rd, 2017			May, 2019
ROS Kinetic Kame	May 23rd, 2016			April, 2021 (Xenial EOL)
ROS Jade Turtle	May 23rd, 2015			May, 2017
ROS Indigo Igloo	July 22nd, 2014			April, 2019 (Trusty EOL)
ROS Hydro Medusa	September 4th, 2013			May, 2015

- In robotics, things are a bit similar.
- Some companies have released robots with a specific version of ROS 1, and while the robot is still on the market, the company will not upgrade and still use and maintain the previous version, also called legacy.
- Thus, the definitive transition in 2025 is going to take a few more years.
- To conclude, for all new learnings, projects, studies, teaching, and startups, ROS 2 is what you need.

Distro	Release date	Poster	Turtle, turtle in tutorial	EOL date
ROS Noetic Ninjemys (Recommended)	May 23rd, 2020			May, 2025 (Focal EOL)
ROS Melodic Morenia	May 23rd, 2018			May, 2023 (Bionic EOL)
ROS Lunar Loggerhead	May 23rd, 2017			May, 2019
ROS Kinetic Kame	May 23rd, 2016			April, 2021 (Xenial EOL)
ROS Jade Turtle	May 23rd, 2015			May, 2017
ROS Indigo Igloo	July 22nd, 2014			April, 2019 (Trusty EOL)
ROS Hydro Medusa	September 4th, 2013			May, 2015

- Why ROS?
- What is ROS?
- When to Use ROS?
- ROS 1 versus ROS 2
- **Prerequisites For Starting With ROS 2**

- It is best that you have some knowledge of the following:
- **Linux command line:** Since we'll be using Ubuntu, being familiar with Linux is mandatory. You don't need to be an expert—you just need to know the basics.
- Many tools in ROS 2 involve the command line, so knowing how to open a Terminal and write basic commands will help you tremendously.

```
mark@linux-desktop: /tmp/tutorial
File Edit View Search Terminal Help
mark@linux-desktop:~$ mkdir /tmp/tutorial
mark@linux-desktop:~$ cd /tmp/tutorial
mark@linux-desktop:/tmp/tutorial$ mkdir dir1 dir2 dir3
mark@linux-desktop:/tmp/tutorial$ mkdir
mkdir: missing operand
Try 'mkdir --help' for more information.
mark@linux-desktop:/tmp/tutorial$ cd /etc ~/Desktop
bash: cd: too many arguments
mark@linux-desktop:/tmp/tutorial$ ls
dir1 dir2 dir3
mark@linux-desktop:/tmp/tutorial$
```

- **Python programming:** The two most common languages for ROS are Python and C++.
- Python is easier to get started with and allows you to prototype things faster. Hence, this is the language we will use for all detailed explanations.
- You need to know Python basics, and object-oriented programming (OOP) is a good plus as ROS 2 is heavily using OOP everywhere.



- **Optional: C++ programming.** Even if the focus of the course is on Python, we are still going to include C++ code for many things we do.
- If you only want to learn Python, you can ignore the C++ code, but of course, if you want to follow C++ instructions, you need C++ basics (better with OOP).
- Learning ROS is already quite challenging, so if you're starting ROS, Linux, and Python from scratch, this could be overwhelming.



- You'll need to have a computer to take this course.
- Regarding the specifications, you don't need anything fancy to get started with ROS 2.
- If you can open a web browser with a few tabs and have a smooth experience, we could say that your computer is good enough to get started.



- Then, later, depending on what you want to do with ROS, you might need a better machine (for example, if you want to simulate multiple robots using lots of sensor and image processing).
- However, it's probably better to wait until you need the extra power to upgrade.
- For now, the most important thing to do is start learning ROS.



- For software requirements, we are going to give you the necessary installation instructions throughout this course.
- All the software we will be using is free to use and open source.
- We will also use Ubuntu 24.04, in which we will run ROS 2.



Thank you for your attention.

Any Questions?

